

SRMS Drona — The Project Explained

Everything about SRMS Drona in easy words. No jargon. If you can read this page, you understand the project.

For the technical version with code, see [`TECHNICAL.md`](#).

1. What is it? (30 seconds)

SRMS Drona is a website where the college's office staff — clerks, accountants, library staff, lab assistants — can do their training online.

Today that training is scattered across WhatsApp forwards, shared drives, and printed notes. Drona puts it in one place. Staff log in, watch short lessons, take a small quiz, and get a certificate. Their department head can see who has finished and who has not.

Think of it as **a training tracker that also teaches the training.**

Who uses it: about 14 people right now — one admin, six department heads, seven staff.

2. The technology, in plain words

We picked tools that are **boring, well-known and hard to break** — because this system has to keep working after we graduate.

What we used	What it does	Why
Python	The programming language we wrote it in	Very common, lots of help available
Django	The main toolkit that builds the website	Comes with security already built in
PostgreSQL	The database — stores all the data	Handles many users at once properly
HTML + CSS	How the pages look	Loads fast even on a slow phone
Argon2	Turns passwords into uncrackable codes	Even if someone steals the database, they get useless codes
Google Gemini	An AI that helps write quiz questions	Saves the admin hours of typing
ReportLab	Makes the PDF certificates	Standard, reliable
Render	The cloud service that hosts the website	Free tier, enough for this scale
GitHub Actions	Automatically tests our code every time we save	Broken code cannot reach the live site

One thing we deliberately did NOT use: React or Node. A lot of projects use them. We didn't, because our users are on phones with slow connections, and pages that come ready-made from the server load far faster. Simpler technology that actually works for the user beats impressive technology that doesn't.

3. The architecture, in plain words

Architecture just means: how the pieces are arranged.

Think of a restaurant kitchen. Orders come in at the front, get prepared in the middle, and the finished plate comes out. Our system works the same way, in **four steps**:

Step	Layer	What it is	In plain words
1	Users	Staff, HOD, admin	The people using it
2	The pages	38 server-rendered screens	What you actually see on screen
3	The brain	Seven modules	The rules — what you are allowed to see
4	The data	PostgreSQL database	Where the facts are kept

Step 1 — Users

Three kinds of person: **staff** (learn), **HOD** (heads a department, sees their own staff), **admin** (runs the whole platform).

Step 2 — The pages (what you see)

The computer asks the server for a page. The server builds the **whole finished page** and sends it over. There is no app to install and nothing that breaks if the internet hiccups. 38 pages, all in English and हिन्दी.

Step 3 — The brain (where the rules live)

Seven separate mini-programs, each in charge of one job — accounts, courses, quizzes, certificates, reports, reminders, and admin records. **Nobody can see someone else's data**, and that rule is checked here, on the server — not by hiding buttons.

Step 4 — The data (where facts live)

The database. Content is stacked neatly:

Category → Course → Module → Lesson

A **Category** is a subject area. A **Course** is like "Fire Safety". A **Module** is a chapter. A **Lesson** is one small piece — a video, a PDF, or some text.

This neat stacking is the important part. Because lessons are counted individually, we can work out exactly how far someone has got — rather than asking them to tick a box.

4. The core modules (the seven jobs)

Think of each app as a department with one responsibility.

Module	Its one job	In plain words
Users	Accounts and logins	Remembers who everyone is, and whether they're staff, a head, or an admin
Courses	Content, progress, video	Holds the lessons, and works out how far each person has got
Quizzes	Tests and AI help	Runs the questions, and helps the admin write new ones
Certificates	Proof of finishing	Makes the PDF certificate and the QR code
Analytics	Reports	Shows how far people got, and lets HR download the data
Notifications	Reminders	Nudges people who haven't finished mandatory training
Management	Record keeping	Logs who did what, for auditing

Why split into seven instead of one big program? Because when something breaks, we know exactly where to look — and the tests for certificates don't have to know anything about reports.

5. The core capabilities (what it can actually do)

For staff

- **Log in** with their **employee ID** — not an email address
- **See their dashboard** — which courses, how far done
- **Watch lessons** — and the video **remembers where you stopped**
- **Switch to हिन्दी** — the whole site, and the choice is remembered
- **Take a quiz** — pass mark is 70%
- **Get a certificate** — with a QR code, downloadable as PDF

For department heads

- Everything above, for themselves
- **See their own department's** progress
- **Add staff to courses** in bulk

For admins

- Everything above
- **Create and edit courses** — add lessons, videos, PDFs
- **Create accounts** and give people their role
- **See reports** across the whole college
- **Download reports** as CSV to put into a spreadsheet
- **Use AI to write quiz questions** from an SOP document

The two cleverest features

1. The video remembers your place. Close the tab, come back tomorrow, and it picks up at the same second. This works because the server understands partial requests — instead of re-downloading the whole video, your browser asks for just the part it needs. That's also why you can drag the timeline smoothly.

2. The QR code is real proof. Every certificate carries a QR code. Anyone scanning it opens a public page that confirms the certificate is genuine — no login needed. But the page shows only the **first name and last initial** (Anant S.), so a certificate can be shown publicly without exposing the person's full name.

6. How someone actually uses it

The whole journey, in plain words:

1. **Admin creates an account** for a staff member, and enrolls them in a course.
 2. **Staff logs in** with employee ID and password.
 3. **Dashboard shows** their courses and how far done.
 4. **They open a course** and read the lessons. Videos remember their place.
 5. **The system counts** what they genuinely finished. It works this out on the server, so nobody can cheat by editing the page.
 6. **They take the quiz.** They need 70% to pass.
 7. **Certificate appears** — but only if they finished ****all the lessons *and* passed the quiz****. Both, not either.
 8. **They download the PDF.** It has a QR code.
 9. **HR checks the report** to see who is done.
-

7. How a certificate is made and checked

This is the feature with the most moving parts, so it is worth explaining twice: first in plain words, then with the actual code.

Part A — In plain words

When you get one. Only after **two** separate things happen:

1. You finish **every lesson** in the course
2. You **pass the quiz**

Both are required. Pass the quiz but skip a lesson and you get a message telling you to finish the lessons. Finish the lessons but fail the quiz and nothing happens. The rule exists so nobody gets a certificate just for turning up.

It is automatic. You do not apply for anything. The moment the second condition is met, the certificate is made and waiting for you.

Each one has a random number. Like a serial number on an engine, but random rather than sequential:

SRMS-CERT-2026-A3F91C2E

Because it is random, nobody can work out someone else's number by trying the next one.

A QR code is printed on it. A QR code is the square pattern a phone camera reads. It contains nothing except one web address with the certificate number in it.

How someone checks it:

Step	What happens
1	They scan the QR code with a phone camera
2	A web page opens — no login, no app, no account
3	The page looks that number up in the real database
4	Found → "This certificate is genuine" , with the course, the date, and a shortened name
5	Not found → "This certificate is not valid" — a clean page, not an error

Why the name is shortened. The check page is open to everybody. If it showed full names, then anyone holding one certificate could open their own page, change the number in the address bar, and slowly read through the college's **entire staff list** — every name, one at a time.

So the page shows only the **first name and the first letter of the surname**: Anant S.

That still proves **whose** certificate it is. But a certificate can be framed, shown to an employer, or held up in public without handing a stranger someone's full identity.

Public to check, private to download. Checking is open to anyone. Downloading the actual PDF requires being logged in as the person who owns it, or as an admin. So the world can confirm a certificate is real, but only the owner can take the document.

Why it cannot be faked. Someone could write their own name on a blank certificate and draw a convincing fake QR code. It still fails, because the QR code only contains a **number**, and that number has to already exist in the real system. A number nobody has heard of returns "not valid". You cannot invent a valid number without actually finishing the course.

Part B — The technical version

The gate that issues it

Only one place in the whole system can create a certificate — the quiz submission handler, `apps/quizzes/views.py`:

```
score_percent = round((correct_count / total_q) * 100, 1)
passed = score_percent >= quiz.passing_score

if course:
    enrollment, _ = Enrollment.objects.get_or_create(
        staff_user=request.user, course=course
    )
    if passed and enrollment.progress_percent >= 100:
        cert = generate_certificate_pdf(request.user, course, request_host=host)
        log_audit(request.user, 'certificate_issued', cert, ...)
    elif passed:
        messages.success(request,
            "You passed the quiz! Complete remaining video/PDF lessons "
            "to receive your certificate.")
```

`passed and progress_percent >= 100` — the two-condition gate. The middle branch is why a user who passed but has lessons left is told exactly what to do.

The certificate number

`apps/certificates/models.py`:

```
def generate_cert_id():
    year = timezone.now().year
    return f"SRMS-CERT-{year}-{uuid.uuid4().hex[:8].upper()}"

class Certificate(models.Model):
    certificate_id = models.CharField(max_length=50, unique=True, default=generate_cert_id)
    staff_user = models.ForeignKey(...)
    course = models.ForeignKey(...)

class Meta:
    unique_together = ('staff_user', 'course')
```

- `uuid4()` — random, not sequential, so IDs cannot be enumerated
- `unique=True` on the ID — gives a unique index for the verification lookup
- `unique_together` — one certificate per person per course, enforced by the database rather than by application code

These IDs are not hashed, and that is deliberate. A hash is one-way, so the verification page could not look the certificate up by ID. The ID is a **public lookup token by design** — which is exactly why the name on the page is masked.

Building the PDF and the QR

apps/certificates/pdf_builder.py:

```
cert, created = Certificate.objects.get_or_create(
    staff_user=staff_user, course=course
)
# <- get_or_create gives the "only one" behaviour

verify_url = _build_verify_url(request_host, cert.certificate_id)
qr = qrcode.QRCode(error_correction=qrcode.constants.ERROR_CORRECT_M)
```

`get_or_create` is what makes a repeat attempt return the existing certificate rather than making a second one.

The verify URL prefers a configured base so a printed certificate always points at the real https site:

```
def _build_verify_url(request_host="127.0.0.1:8000", cert_id=""):
    base = (getattr(_s, 'SRMS_BASE_URL', '') or '').rstrip('/')
    if base:
        if '://' not in base:
            base = f"https://{base}"
        return f"{base}/verify/{cert_id}/"
    scheme = 'https' if ('railway.app' in host or 'srms.ac.in' in host) else 'http'
    return f"{scheme}://{host}/verify/{cert_id}/"
```

The QR is generated **before** the PDF so it can be drawn onto the page. `ERROR_CORRECT_M` means the code still scans if the certificate gets scuffed, folded or slightly covered in a frame.

The public verification page

apps/certificates/views.py — note there is no `@login_required`:

```
def verify_certificate_view(request, cert_id):
    try:
        certificate = Certificate.objects.select_related(
            'staff_user', 'course').get(certificate_id=cert_id)
        is_valid = True
    except Certificate.DoesNotExist:
        certificate = None
        is_valid = False
    ...
```

A made-up ID renders the "not valid" template. It does **not** 500, and it does not return a different response shape — so the page cannot be used to discover which IDs are real.

The name masking:

```
def _masked_name(user):
    """First name plus last initial.
```

```
    The verification page is public by design -- it is what the QR code on a
    printed certificate resolves to -- so it must confirm a match without
    publishing a full staff roster to anyone who holds a certificate ID.
    """
```

```
    first = (user.first_name or "").strip()
    last = (user.last_name or "").strip()
```

```

if first and last:
    return f"{first} {last[0].upper()}"
...
return "SRMS staff member"

```

The protected download

Verification is public, but the document is not. `download_certificate_pdf`:

```

@login_required
def download_certificate_pdf(request, cert_id):
    qs = Certificate.objects.filter(certificate_id=cert_id)
    if not request.user.is_manager:
        qs = qs.filter(staff_user=request.user) # scoped to the owner
    certificate = get_object_or_404(qs)

    if not certificate.pdf_file or not os.path.exists(certificate.pdf_file.path):
        generate_certificate_pdf(...) # self-healing
        certificate.refresh_from_db()

```

Three things worth noting: it requires a login; a non-manager is filtered to their **own** certificates so one user cannot download another's; and a missing file is **regenerated** rather than returning a dead link.

The routes

URL	Logged in?	Who can use it
<code>/verify/<cert_id>/</code>	No	Anyone — this is what the QR opens
<code>/certificates/</code>	Yes	Your own certificate list
<code>/certificates/<cert_id>/download/</code>	Yes	The owner, or any manager

Note the verify route sits at the top level, deliberately outside `/certificates/`, which is behind a login.

Why the whole design holds up

Attack	What stops it
Forging a certificate by hand	The QR holds a number; an unknown number returns "not valid"
Guessing someone else's number	<code>uuid4()</code> is random and long — not enumerable
Getting two certificates for one course	<code>unique_together</code> in the database
Self-reporting 100% progress	Progress is derived from <code>LessonProgress</code> rows, not accepted from the client
Passing the quiz without doing the work	Watch time is accumulated server-side and capped; the client's <code>completed</code> flag is discarded for video
Using a public check page to harvest staff names	Only first name + last initial is ever rendered
Downloading someone else's certificate	<code>@login_required</code> plus an owner filter on the queryset

Every one of those has a test in the suite, so the guarantee cannot quietly rot.

8. Three things that make it trustworthy

Progress is worked out by the server, not claimed by the user

If you change something in your browser, the server ignores it. You cannot mark yourself 100% done — the server counts the lessons you actually completed.

Watching is measured, not assumed

You could drag a video to the end in one click. So the system counts the time it has actually **sent** you, and ignores the "finished" flag your browser sends for video lessons. One heartbeat can only ever add 30 seconds, and the total can never go past the video's real length.

The tests never sleep

203 automated tests run **every single time** anyone saves code. If something breaks, the live website cannot be updated. That is why we can say the system works.

9. Three real bugs we found and fixed

We didn't just take a checklist and tick it off. We tried to **break our own system**, and found three genuine problems:

#	What was wrong	How we found it	Fix
1	Someone could upload a file that ran code in other people's browsers	Uploading a file that lied about its type	Now the filename and the file type must both agree, and the server decides the real type
2	Someone could try millions of passwords without getting blocked	The system trusted a header the user could fake	Now it uses the real connection address it cannot be tricked into
3	Real login details were published in our public code repository	Reading our own README	All 14 accounts got new unique passwords, and the details moved to a secure password manager

The pattern: all three were found by **testing a control against a real attacker**, not by reading that the control existed. And each one now has an automated test, so it cannot come back.

10. What is honest about the limits

We think a project that admits its gaps is stronger than one that claims perfection. So:

- **Emails are not being sent yet.** The reminder feature is fully built, but the college has not given us email server access, so no reminders go out. The on-screen reminders are also switched off by default until that is fixed.

- **We have not tested it under heavy load.** We know it works with the people using it now. We do not claim it is proven for hundreds at once.
- **Videos are stored on the server's own disk.** Fine now, but before real scale it should move to proper cloud storage.
- **The screen reader / accessibility pass is not finished.**

11. The short version

SRMS Drona is a training website for college office staff, built with Django and PostgreSQL, hosted free in the cloud.

It is organised in four layers — users, pages, rules, database — split into seven focused modules so problems are easy to find and test.

It is bilingual, remembers where you stopped watching, and issues QR-verifiable certificates only after you finish the lessons and pass the quiz.

Its progress is calculated on the server, so it cannot be faked. It runs 203 automatic tests, has no known security problems, and is live at dronav2.onrender.com.

Where to look next

If you want...	Open
The simple overview	This page
Every feature explained	`PROJECT-GUIDE.md`
How the code works, in detail	`TECHNICAL.md`
What to say in a presentation	`presentation/SCRIPT.md`
The full written report	`report/report3.0.pdf`
The actual code	< https://github.com/dgexplores/DRONA >
The live website	< https://dronav2.onrender.com >